

Hamilton-Jacobi-Bellman Guided Deep Reinforcement Learning for Optimal Liquidation Under Time-Varying Liquidity

Alexander Bowari
abowari2@illinois.edu

Abstract

Optimal liquidation seeks to determine a trading strategy that minimizes execution costs, market risk, and inventory exposure. Typically, large institutional orders cannot be executed instantaneously without incurring those risks. The optimal liquidation problem addresses this challenge by determining how a position should be sold over time while balancing profitability and risk. In this paper, we investigate the connection between stochastic optimal control and reinforcement learning through the development of an intelligent liquidation agent that adjusts a derived formula for trading velocity. We begin by formulating the problem within the Almgren-Chriss framework and derive the associated Hamilton-Jacobi-Bellman equation using dynamic programming principles. An ansatz transformation is then used to simplify the resulting nonlinear optimization problem and reveal the structure of the optimal trading strategy. Building upon these theoretical foundations, we can observe a reinforcement learning agent that learns liquidation policies in a simulated market environment. Our results demonstrate that reinforcement learning can help closely approximate optimal liquidation strategies while benefiting from the mathematical guidance provided by classical control methods, offering a unified perspective on reinforcement learning and stochastic optimization in financial markets.

1 Introduction

Financial bodies frequently face the challenge of liquidating large asset positions without significantly affecting market prices. While executing a single large order may complete a sale quickly, it can generate substantial market impact, cause unfavorable price movements, and reduce expected revenue. Conversely, distributing trades over time can mitigate immediate price impact but exposes the trader to uncertainty arising from stochastic price fluctuations. Balancing these competing objectives has motivated decades of research in optimal execution and algorithmic trading.

Classical approaches, most notably the Almgren-Chriss framework, formulate this problem as stochastic optimal control. Through dynamic programming and the Hamilton-Jacobi-Bellman equation, one can derive a trading velocity that optimally trades off execution cost and inventory risk. Despite their elegance, such solutions depend on assumptions regarding market behavior that may not be held in practice.

In this work, we investigate the intersection of stochastic control and reinforcement learning for optimal liquidation. We derive the theoretical structure of the liquidation problem from first principles, analyze its Hamilton-Jacobi-Bellman formulation, and demonstrate how reinforcement learning can be used to approximate optimal execution policies. The resulting framework combines the mathematical rigor of classical control theory with the flexibility of data-driven decision making to answer one question: Can a reinforcement learning agent assist a derived trading velocity from a Hamilton-Jacobi-Bellman optimal liquidation strategy under time-varying liquidity?

2 Calculating a Value System

The central challenge of optimal liquidation arises from two competing sources of cost. Executing shares too rapidly places substantial pressure on available market liquidity, causing the traders to move the market against themselves. Conversely, executing too slowly leaves a large inventory exposed to unpredictable price fluctuations, increasing the risk that the asset value deteriorates before liquidation is complete. Put simply, the answer to optimal liquidation is finding a favorable median between these two risks. The Almgren-Chriss framework captures this trade-off through two distinct components.

The first is the market impact, where n is the liquidity parameter and v_t is the trading velocity:

$$C_{\text{impact}} = nv_t^2. \quad (1)$$

This quadratic form reflects a fundamental feature of market microstructure: the cost of trading accelerates as speed increases, because when a trader executes at a certain velocity, they consume liquidity faster than the market can naturally replenish it. Market makers and other participants respond by widening spreads and adjusting quotes adversely, so each additional unit traded at higher speed becomes progressively more expensive.

The second is the inventory risk cost, where λ is the risk-aversion parameter, σ is the asset volatility, and X_t is the remaining inventory:

$$C_{\text{risk}} = \lambda\sigma^2 X_t^2. \quad (2)$$

The remaining inventory enters quadratically because risk scales with variance. For instance, a position twice as large quadruples the exposure to adverse price movements. The volatility σ translates price uncertainty into inventory cost, while λ governs the trader's willingness to accept this risk relative to market impact: a higher λ demands faster execution despite greater impact, a lower λ accepts slower or cheaper execution at the cost of prolonged exposure.

It's very important to note that the Almgren-Chriss framework assumes n , λ , and σ are fixed constants, an assumption routinely violated in actual practice. Liquidity fluctuates with time-of-day and market events, volatility is time-varying and regime-dependent, and a static calculation of either parameter can be severely suboptimal when conditions shift mid-execution.

3 Deriving the Hamilton-Jacobi-Bellman Equation

Let X_t be the remaining inventory at time t , evolving according to the trader's chosen velocity v_t :

$$\frac{dX_t}{dt} = -v_t, \quad dX_t = -v_t dt. \quad (3)$$

The trader's objective is to minimize the total expected cost over the liquidation horizon 0 to T , which we will call cost J and can denote as an integral of the two expected costs from the beginning of the trading period to the end:

$$\text{Cost} = J(x(t), u(t), t_0, t_f), \quad (4)$$

$$J = \mathbb{E} \left[\int_0^T (nv_t^2 + \lambda\sigma^2 X_t^2) dt \right]. \quad (5)$$

Now we can define the value function as the minimum achievable cost from a first given state, with a terminal condition that $V(T, X)$ is equal to zero, since no cost is incurred once liquidation ends:

$$V(t, X) = \min_v J(t, X), \quad (6)$$

$$V(t, X) = \min_v \mathbb{E} \left[\int_0^T (nv_t^2 + \lambda\sigma^2 X_t^2) dt \right]. \quad (7)$$

Given the established inventory state, we can start taking the total time derivative of the value function along the state trajectory. It is also important to know that derivatives are transposed when multiplied due to dot product rules, and that the condition T delineating the final time was introduced to show that the variable has no effect when finding the derivative of V . This expression captures the full rate of change of the value function: the first term reflects the passage of time at fixed inventory, while the second reflects the change in value induced by the depletion of X under control v :

$$\frac{d}{dt}(V(X_t, t, T)) = \frac{\partial V}{\partial t} \frac{dt}{dt} + \frac{\partial V}{\partial x} \frac{dx}{dt} + \frac{\partial V}{\partial T} \frac{dT}{dt}, \quad (8)$$

$$\frac{dV}{dt} = \frac{\partial V}{\partial t} + \left(\frac{\partial V}{\partial x} \right)^T \frac{dx}{dt}. \quad (9)$$

The key structural property of the value function is Bellman's principle of optimality: the optimal policy from any intermediate state must itself be optimal for the remaining subproblem. We can show this by taking the derivative of the second definition of our value function, which was defined as the minimum of the total cost:

$$L(X_t, v) = nv^2 + \lambda\sigma^2 X_t^2, \quad (10)$$

$$\min_v \frac{d}{dt} \left[\int (L(X_t, v)) dt \right] = \min_v [L(X_t, v)]. \quad (11)$$

Now we can set the two definitions of our value function equal to each other, remembering that the rate at which inventory X depletes with respect to time is equal to the trading velocity:

$$\min_v [L(X_t, v)] = \frac{\partial V}{\partial t} + \left(\frac{\partial V}{\partial x} \right)^T \frac{dx}{dt}, \quad (12)$$

$$\min_v [L(X_t, v)] = \frac{\partial V}{\partial t} + \left(\frac{\partial V}{\partial x} \right)^T v, \quad (13)$$

$$-\frac{\partial V}{\partial t} = \min_v \left[\left(\frac{\partial V}{\partial x} \right)^T v + L(X_t, v) \right]. \quad (14)$$

Subbing back in our costs from earlier, we obtain a derivation of the Hamilton-Jacobi-Bellman equation in terms of our own variables:

$$-\frac{\partial V}{\partial t} = \min_v \left[\left(\frac{\partial V}{\partial x} \right)^T v + nv^2 + \lambda\sigma^2 X_t^2 \right]. \quad (15)$$

This partial differential equation must hold for all $(X, t) \in \mathbb{R} \times [0, T)$ subject to the terminal condition that $V(X(T), T)$ is equal to zero. It fully characterizes the optimal liquidation policy: any control velocity that achieves the minimum on the right-hand side at every point (X, t) is the globally optimal trading velocity, and the corresponding V is the minimum cost the trader can guarantee from that state onward.

4 Deriving the Optimal Liquidation Policy via the Hamilton-Jacobi-Bellman Framework

We seek a principled method for determining the optimal trading velocity at every point in the liquidation. The Hamilton-Jacobi-Bellman framework provides precisely this: a PDE whose solution encodes the minimum

achievable cost from every possible state, and whose minimizer at each point yields the globally optimal control.

By Bellman's principle of optimality, the value function representing the minimum cost achievable from inventory X_t can be decomposed into the immediate cost incurred over an infinitesimal interval, plus the optimal cost from the resulting state. Doing this gives:

$$V(X(t), t) = \min_v \left[\int_t^{t+dt} (nv^2 + \lambda\sigma^2 X^2) ds + V(t + dt, X - v dt) \right]. \quad (16)$$

To make this expression actually tractable, we are able to expand the last term—representing the value of being in that reduced-inventory state one instant later—as a first-order Taylor series around (X, t) :

$$V(t + dt, X - v dt) = V(t, X) + \frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial x} dx + \dots, \quad (17)$$

$$V(t + dt, X - v dt) = V(t, X) + \frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial x} (-v dt). \quad (18)$$

The higher-order terms vanish as dt approaches zero. Substituting this expansion back into the Bellman equation, the term $V(t, X)$ cancels from both sides, and then for further simplification we divide through by dt :

$$V = \min_v \left[(nv^2 + \lambda\sigma^2 X^2) dt + V + \frac{\partial V}{\partial t} dt - v \frac{\partial V}{\partial x} dt \right], \quad (19)$$

$$0 = \min_v \left[(nv^2 + \lambda\sigma^2 X^2) dt + \frac{\partial V}{\partial t} dt - v \frac{\partial V}{\partial x} dt \right], \quad (20)$$

$$0 = \min_v \left[nv^2 + \lambda\sigma^2 X^2 + \frac{\partial V}{\partial t} - v \frac{\partial V}{\partial x} \right]. \quad (21)$$

Separating the terms that carry no dependence on trading velocity, we arrive once again at the Hamilton-Jacobi-Bellman equation:

$$-\frac{\partial V}{\partial t} = \min_v \left[nv^2 + \lambda\sigma^2 X^2 - v \frac{\partial V}{\partial x} \right]. \quad (22)$$

This partial differential equation states that the rate at which the value function decreases over time must equal the minimum instantaneous cost achievable by the trader at that state, a precise mathematical expression of the intuition that the optimal policy exhausts all available opportunities to reduce cost at every moment.

Now, we can solve for optimal liquidation. The right-hand side of the Hamilton-Jacobi-Bellman equation is a convex quadratic in v , which guarantees a unique global minimizer. We obtain the optimal control v by differentiating with respect to v and setting the result to zero:

$$\frac{d}{dv} \left(nv^2 - v \frac{\partial V}{\partial x} + \lambda\sigma^2 X^2 \right) = 0, \quad (23)$$

$$2nv - \frac{\partial V}{\partial x} = 0, \quad (24)$$

$$2nv = \frac{\partial V}{\partial x}, \quad (25)$$

$$v = \frac{\partial V / \partial x}{2n}. \quad (26)$$

Substituting the expression for the optimal trading velocity into the Hamilton-Jacobi-Bellman equation

eliminates the minimization and yields a single partial differential equation governing the value function:

$$-\frac{\partial V}{\partial t} = \lambda \sigma^2 X^2 - \frac{\left(\frac{\partial V}{\partial x}\right)^2}{4n}. \quad (27)$$

This equation fully characterizes the optimal liquidation policy, any solution V to this equation immediately yields the optimal trading velocity at every state. However, as a nonlinear partial differential equation, it does not admit a general closed-form solution without further structural assumptions. In the following section, we exploit the quadratic structure of the cost functions to propose an ansatz of the form which reduces this equation to a tractable system of ordinary differential equations.

5 Using the Ansatz Trick to Find the Value Function

Continuing from the reduced partial differential equation, we solve it explicitly via the ansatz method. Inspecting the structure of this equation, both the running cost and the market impact term are even in X , which suggests that the value function itself should be quadratic in inventory. We therefore propose an ansatz of the form:

$$V(X, t) \approx a(t)X^2. \quad (28)$$

Moreover, given this assumption, we can compute the value function's necessary partial derivatives:

$$\frac{\partial V}{\partial x} = 2a(t)X, \quad \frac{\partial V}{\partial t} = a'(t)X^2. \quad (29)$$

This ansatz trick reflects the economic structure of the problem. Since both sources of cost scale quadratically in X , it is natural that the minimum achievable cost from any state should also scale quadratically in the remaining inventory. Now we can begin substituting into our reduced partial differential equation:

$$0 = \frac{\partial V}{\partial t} + \lambda \sigma^2 X^2 - \frac{\left(\frac{\partial V}{\partial x}\right)^2}{4n}, \quad (30)$$

$$0 = a'(t)X^2 + \lambda \sigma^2 X^2 - \frac{4X^2 a(t)^2}{4n}. \quad (31)$$

Now, we can separate powers of X by factoring out and canceling the term, leaving us with the Riccati equation:

$$a'(t) = -\lambda \sigma^2 + \frac{4a(t)^2}{4n}. \quad (32)$$

Solving this ordinary differential equation yields the closed-form optimal trading schedule, which we derive in the following section.

6 Solving the Riccati Equation

To solve for an expression that yields an optimal trading velocity, we can start by expressing it as an ordinary differential equation:

$$\frac{da}{dt} = -\lambda \sigma^2 + \frac{4a(t)^2}{4n}. \quad (33)$$

From this equation, we can employ separation of variables and integrate with partial fractions to find a solution:

$$k = \sqrt{n\sigma^2\lambda}, \quad (34)$$

$$\frac{da}{dt} = -\lambda \sigma^2 + \frac{4a(t)^2}{4n}, \quad (35)$$

$$\frac{da}{\frac{1}{n}a^2 - \lambda\sigma^2} = dt, \quad (36)$$

$$\int \frac{n da}{a^2 - k^2} = \int dt, \quad (37)$$

$$\frac{n}{2k} \int \left(\frac{1}{a-k} + \frac{1}{a+k} \right) da = \int dt, \quad (38)$$

$$\frac{n}{2k} \ln \left(\frac{a+k}{a-k} \right) = t + C. \quad (39)$$

From this expression, we can exponentiate and substitute variables in to find the function a , ultimately giving us an expression we can further simplify with the tangent function:

$$\frac{n}{2k} \ln \left(\frac{a+k}{a-k} \right) = t + C, \quad (40)$$

$$\frac{a(t) + k}{a(t) - k} = A e^{\frac{2kt}{n}}, \quad (41)$$

$$a(t) = k \frac{1 - e^{\frac{2k(t-T)}{n}}}{1 + e^{\frac{2k(t-T)}{n}}}, \quad (42)$$

$$a(t) = -k \tanh \left(k \frac{T-t}{n} \right). \quad (43)$$

This holds true where $k = \sqrt{n\lambda\sigma^2}$. Since $t \leq T$, the argument of $\tanh$ is non-negative and $a(t) \leq 0$ for all $t \in [0, T]$, which is consistent with the value function being non-positive, reflecting that the optimal cost is a reduction relative to uncontrolled liquidation.

Recovering the Optimal Trading Velocity

Recall that the optimal control was found to be:

$$v = \frac{\partial V / \partial x}{2n}. \quad (44)$$

Since the value function is equal to $a(t)X^2$, the partial derivative of the value function with respect to x is equal to $2a(t)X$, therefore:

$$v(X, t) = \frac{2a(t)X}{2n}. \quad (45)$$

Substituting the closed-form solution for $a(t)$, we obtain a formula for the trading velocity for optimal liquidation:

$$v(X, t) = -\sqrt{\frac{\lambda\sigma^2}{n}} \tanh \left(\sqrt{\frac{\lambda\sigma^2}{n}} (T-t) \right) X. \quad (46)$$

7 Deep Residual Reinforcement Learning for Trading Velocity Adjustment

The trading velocity derived through the Hamilton-Jacobi-Bellman framework provides a theoretically optimal liquidation schedule under the assumptions of the Almgren-Chriss model. However, real markets rarely satisfy these assumptions exactly. Liquidity varies throughout the trading day, volatility changes in response to news and market events, and the behavior of other market participants introduces dynamics that are difficult to

model analytically. As a result, a trading schedule that is optimal under a simplified mathematical framework may become suboptimal when deployed in practice.

To address this limitation, we introduce a reinforcement learning agent that operates in conjunction with the analytically derived trading policy. Rather than learning liquidation from scratch, the agent observes the market environment and learns how to adjust execution decisions in response to changing conditions. The Hamilton-Jacobi-Bellman solution therefore serves as a mathematically grounded baseline, while reinforcement learning provides the flexibility required to adapt to market behavior that cannot be fully captured by closed-form models.

$$v_{final} = v_{HJB} + v_{RL} \quad (47)$$

$$v_t = v_{HJB}(X_t, t) + v_{\theta}(s_t) \quad (48)$$

A viable question still remains unanswered: how would a deep reinforcement learning bot actually function to correct the behavior of the Hamilton-Jacobi-Bellman derived solution for an optimal trading velocity? First, let us define a state s at time t :

$$s_t = [X_t, T_{final} - t_{initial}, \text{market features}] \quad (49)$$

Having established the state representation at time t , we now introduce a neural network responsible for generating corrections to the analytically derived trading velocity. We define the neural network parameterized by weights and biases across several hidden layers. Each layer is forward propagated according to:

$$h^{(k)} = \sigma\left(W^{(k)} h^{(k-1)} + b^{(k)}\right), \quad k = 1, \dots, L \quad (50)$$

W represents the multiplied weight vector, b represents the added bias, and the activation function is delineated as sigma, represented by the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (51)$$

The output layer applies a linear projection to the final hidden representation to produce a single scalar, which is the learned correction to the trading velocity:

$$\Delta v_t = W^{(L+1)} h^{(L)} + b^{(L+1)} \quad (52)$$

At each time step t , we define the discounted cumulative return as the sum of future rewards weighted by a discount factor:

$$R_t = \sum_{k=t}^T \gamma^{k-t} r_k \quad (53)$$

where gamma is the discount factor between 0 and 1, r is the reward received at step k , and T is the terminal time horizon. The discount factor penalizes rewards received further in the future, ensuring the sum remains bounded and reflecting the time preference of the liquidation objective.

The goal of the learning procedure is to find the network parameters θ that maximize the expected discounted return over the distribution of trajectories induced by the policy. Formally, our objective is defined as maximizing J based on the function's parameters:

$$J(\theta) = \mathbb{E}[R_t] \quad (54)$$

Defining the Critic

We define the critic as the action-value function under the current velocity policy v , which estimates the expected discounted return when taking action a in state s and thereafter following policy v . This is given by the Bellman expectation equation:

$$Q^v(s_t, a_t) = \mathbb{E}[r_t + \gamma Q^v(s_{t+1}, a_{t+1})] \quad (55)$$

We approximate the action-value function using a neural network with learnable parameters ϕ , denoted as:

$$Q_\phi(s_t, a_t) \quad (56)$$

To train this network, we first construct the temporal difference target at each time step. Then, the critic loss is then defined as the squared deviation between this target and the current network estimate

$$y_t = r_t + \gamma Q_\phi(s_{t+1}, v(s_{t+1})) \quad (57)$$

$$\mathcal{L}_Q = (y_t - Q_\phi(s_t, a_t))^2 \quad (58)$$

Backpropagating this loss through the network, the gradient with respect to the critic parameters follows by the chain rule:

$$\frac{\partial \mathcal{L}_Q}{\partial \phi} = \frac{\partial \mathcal{L}_Q}{\partial Q_\phi} \cdot \frac{\partial Q_\phi}{\partial \phi} \quad (59)$$

The critic parameters are then updated via gradient descent,

$$\phi \leftarrow \phi - \alpha \frac{\partial \mathcal{L}_Q}{\partial \phi} \quad (60)$$

Updating the Actor via Policy Gradient

To optimize the actor, we define the objective as the expected critic value under the current policy,

$$J(\theta) = \mathbb{E}[Q_\phi(s, v_\theta(s))] \quad (61)$$

where the subscript θ denotes that the policy is a neural network fully determined by the parameter vector θ , and the critic evaluates the expected discounted return associated with executing the velocity prescribed by that policy from state s . Maximizing this objective requires differentiating through the critic with respect to the actor parameters. Applying the chain rule, the gradient of J decomposes into the sensitivity of the critic to changes in the action, composed with the sensitivity of the policy output to changes in θ :

$$\nabla_\theta J(\theta) = \frac{\partial Q_\phi}{\partial v} \cdot \frac{\partial v}{\partial \theta} \quad (62)$$

Taking the expectation over the distribution of states visited under the current policy, this becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_v Q_\phi(s, v_\theta(s)) \cdot \nabla_\theta v_\theta(s)] \quad (63)$$

This expression identifies the direction in parameter space along which the actor must move in order to produce actions that the critic assigns higher value. The actor parameters are updated via gradient ascent: taking steps up the function instead of down because the goal is to maximize J :

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \quad (64)$$

where α is the actor learning rate.

Synthesis and Significance

Taken together, the equations developed in this section define a complete actor-critic reinforcement learning agent that operates directly on the structure of the Hamilton-Jacobi-Bellman framework. The actor, parameterized by θ , maps the market state to a residual correction to the analytically derived trading velocity. The critic, parameterized by ϕ , evaluates the quality of that correction by approximating the action-value function through recursive Bellman estimation. The two networks are trained in conjunction with one another: the critic is driven toward consistency with observed rewards via minimization of the temporal difference loss, while the actor is pushed toward policies that the critic assigns higher expected return via gradient ascent through the critic itself.

What distinguishes this framework from classical optimal liquidation is that the agent does not require explicit specification of things such as risk aversion, asset volatility, or market impact parameters in order to produce a well-adapted trading policy. These quantities are implicitly encoded in the reward signal received during interaction with the environment, and the actor learns to account for them through experience rather than through analytical prescription. The Hamilton-Jacobi-Bellman solution that is derived provides a principled structural prior that anchors the policy in theory, while the residual network assumes responsibility for correcting the systematic errors that arise when the true market dynamics deviate from the idealized assumptions underlying that solution. The result is a hybrid optimal liquidation strategy that is simultaneously grounded in financial theory and robust to real-world specifications.

References

- [1] Almgren, R., & Chriss, N. (2001). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5–39.
- [2] Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- [3] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [4] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 387–395.
- [5] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. *International Conference on Learning Representations (ICLR)*.
- [6] Fleming, W. H., & Soner, H. M. (2006). *Controlled Markov Processes and Viscosity Solutions* (2nd ed.). Springer.